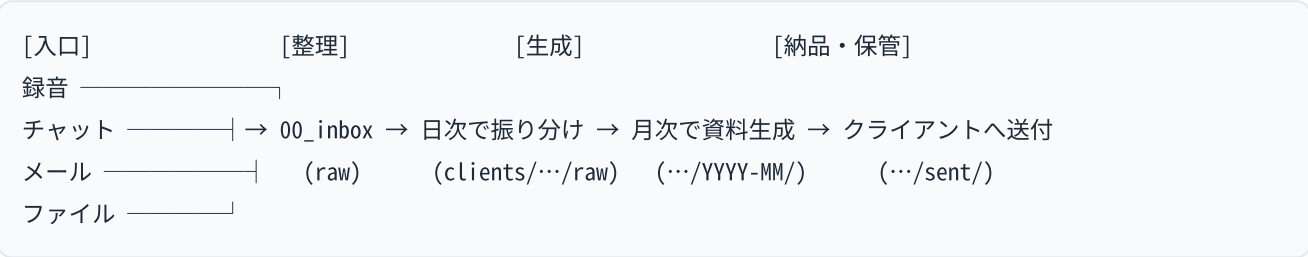


コンサル業務記録ツール 設計書 v0

作成日: 2026-09-14 対象: コンサルタント個人のPC内で、実際の業務記録を蓄積し、議事録・提案書・月次レポートを生成してストックする仕組み 前提: Obsidian（記録の器）＋ Claude Code（整理・生成エンジン）＋数本のPythonスクリプト 大原則: ソース（録音・チャット・メール・ファイル）に遡れないことは書かない。書けない項目は空欄のまま残す。

0. 全体の流れ



人が触るのは3ヶ所だけ。 1. 会議のたびに録音を inbox に落とす（または自動） 2. 日次の振り分け結果を1分で確認する 3. 月次の生成物をレビューして送る

1. 情報の入口（どこから何を取るか）

種別	取得方法	形式	頻度
オンライン会議	tl;dv 等で録音→文字起こし→エクスポート。または Meet/Zoom の録画をローカルWhisperで文字起こし	.md（話者・タイムスタンプ付き）	会議ごと
対面会議	ボイスレコーダー or iPhone録音 → ローカルWhisper（MacWhisper / whisper.cpp）	.md	会議ごと
チャット（Slack）	標準エクスポート（JSON） → 整形スクリプトで .md	.md（日別）	週次
チャット（LINE）	トーク履歴の送信（.txt） → 整形スクリプト	.md（日別）	週次
メール（Gmail）	Gmail API か Google Takeout → 整形スクリプト。クライアントのドメインでフィルタ	.md（1通1ファイル）	週次
作成した資料	作業フォルダを監視、または月末に手動でコピー	元ファイル＋ .md のメモ	随時
電話・移動中の思いつき	Obsidian のデイリーノートに一行メモ（日付・相手・要点）	.md	随時

入口で必ず付ける情報（ファイル名で持つ）

YYYY-MM-DD_<種別>_<相手or件名>.md

例: 2026-09-14_meeting_A社_月次定例.md

2026-09-12_slack_A社_在庫の相談.md

2026-09-10_mail_B社_見積の件.md

種別: meeting / slack / line / mail / doc / memo

入口の注意

- 文字起こしはローカル実行を基本にする（音声はPCの外に出さない）
- クライアントの機密を含むので、クラウド同期フォルダには置かない（Obsidian の vault を iCloud/Dropbox 配下にしない）
- 「電話・思いつき」のメモが一番落ちやすく、一番価値がある。デイリーノートに一行書く習慣が前提になる

2. フォルダ構成

```
consulting-vault/
├── 00_inbox/                ← 全ての入口。ここに落とすだけ
├── 01_clients/
│   ├── A社/
│   │   ├── _profile.md     ← 契約内容・月額・担当者・目標KPI・注意点
│   │   ├── raw/            ← 振り分け済みの生データ（種別問わず日付順）
│   │   ├── 2026-09/
│   │   │   ├── worklog.md  ← 稼働ログ（自動生成）
│   │   │   ├── minutes/    ← 議事録（会議ごと）
│   │   │   ├── proposals/  ← 提案書
│   │   │   ├── qa/         ← チャット相談のQ&A記録
│   │   │   ├── report.md   ← 月次レポート
│   │   │   └── sent/       ← 実際に送付した最終版（docx/pdf）＋送付記録
│   │   └── 2026-10/
│   └── B社/
├── 02_templates/           ← 各資料のテンプレート（Markdown）
├── 03_scripts/             ← Python整形スクリプト
├── 04_logs/                ← 処理ログ（何をいつ振り分けた・生成したか）
└── CLAUDE.md               ← Claude Code への指示（ルール・禁止事項・テンプレートの場所）
```

ルール

- raw/ は絶対に編集しない。ソースとして固定する
- YYYY-MM/ 配下は全て生成物。作り直し可能

- sent/ だけが「納品した事実」。ここに入ったものは触らない
- _profile.md は利用者が手で書く。これが生成の前提知識になる

3. 各資料の作り方

3-1. 稼働ログ（worklog.md） — 全ての土台

raw/ の1ファイル1行で、月の全活動を一覧にする。

```
| 日付 | 種別 | 作業分類 | 相手 | 要点 | 所要目安 | ソース |
|---|---|---|---|---|---|---|
| 09-12 | slack | 相談対応 | A社 担当者 | 在庫の持ち方について助言 | 20分 | 2026-09-12_slack_A社_在庫の相談.md |
```

- 作業分類: 準備 / 会議 / 調査 / 提案 / 相談対応 / フォロー / 資料作成
- 所要目安は「録音の長さ」「メッセージ数」から機械的に出す。推定であることを列名に明記
- 月末に合計を出す（会議○時間、相談対応○件、提案○本）

3-2. 議事録（minutes/） — 会議ごと

テンプレート:

```
# 議事録: <クライアント> <会議名> <日付>
sources: [<文字起こしファイル名>]
## 参加者
## 議題
## 決定事項
## 提案した内容（発言ベース）
## 宿題・次回まで
## 未確定・要確認
```

- 「提案した内容」は文字起こしの発言から起こす。ここが「口頭で終わっていたものを成果物にする」場所
- 文字起こしにない決定事項は書かない

3-3. 提案書（proposals/） — 会議・チャットで提案したものを文書化

```
# 提案: <テーマ> <日付>
sources: [議事録 or チャットのファイル名]
## 背景（クライアントの状況: _profile.md と raw から）
## 提案内容（発言・メッセージから）
## 期待効果（試算がある場合のみ。ない場合は「要試算」と書く）
## 実施ステップ
## 前提・リスク
```

- 期待効果の数字は、根拠がソースにあるときだけ。AIが作った試算は「試算（生成時点の仮定：～）」と明記

3-4. Q&A記録（qa/） — チャット相談のまとめ

チャットで答えた相談を「質問→回答→その後」の形に整理。月に何件対応したかが見える。

3-5. 月次レポート（report.md） — クライアントに送る本体

```
# 月次レポート <クライアント> <YYYY-MM>
sources: [worklog.md, minutes/*, proposals/*, qa/*]
## 今月の稼働サマリー（worklog の集計）
## 会議の要点（各議事録の決定事項）
## 提案と進捗
## 相談対応（Q&Aの件数と主なテーマ）
## KPIの推移（_profile.md の目標KPI と、クライアントから受け取った数値がある場合）
## 来月の予定
```

3-6. 本物の成果物（任意・強く推奨）

- 業界動向レポート、競合調査、次月提案 など、AIで下書き→利用者がレビュー→クライアントに送付
- これは「記録」ではなく「新しい業務」。送った時点で実態になる
- 生成時は Web検索を使い、出典URLを残す

生成ルール（CLAUDE.md に書く）

1. sources: に列挙したファイル以外の情報を書かない
2. 書けない項目は空欄か「要確認」。それらしい文で埋めない
3. 数字は、ソースに書いてあるものだけ。試算は「試算」と明記し仮定を添える
4. 文体はクライアントごとに _profile.md の指定に従う
5. 生成物の末尾に「本資料は以下の記録から作成: …」を自動で付ける

4. 発火（いつ、何が動くか）

タイミ ング	動くもの	やること
会議直 後	利用者（手動）	録音を 00_inbox/ に置く。tl;dv なら自動エクスポート
毎日 22:00	03_scripts/inbox_sort.py (launchd/cron)	inbox の文字起こし・エクスポートを整形し、クライアント別 raw/ に振り分け。判定できないものは inbox に残して通知
毎日 22:05	Claude Code /daily-check	振り分け結果を要約して1通知（「A社に会議1件、B社にSlack3件を 振り分け。未振り分け1件」）
毎週 金 曜	利用者（手動、5分）	Slack / LINE / Gmail のエクスポートを inbox に投入
毎月 1日	Claude Code /monthly	前月の worklog / minutes / proposals / qa / report を全クライア ント分生成
毎月 1～ 3日	利用者（手動）	生成物をレビュー・修正 → sent/ に確定版を置く → クライアント に送付
送付時	03_scripts/send_log.py	送付日時・宛先・添付ファイルを sent/送付記録.md に追記

発火の実装

- 定時実行: macOS なら launchd、Windows なら タスクスケジューラ。n8n は入れない
- Claude Code の定期実行: スクリプトから `claude -p "/monthly"` のようにヘッドレス実行（コマンドの正
確な形式は導入時に公式ドキュメントで確認）
- 通知: macOS 通知 か 自分宛Slack。それ以上は作らない

最小構成での開始

最初の1ヶ月は発火を全部手動にする。「会議があったら inbox に置く」「月末に /monthly を打つ」の2つだ
け。自動化は運用が回ってから。

5. 管理（壊れないための運用）

5-1. 品質チェック（月次レビューで見る3点）

- 生成物の各段落が `sources:` のどれかに対応しているか（抜き打ちで3ヶ所）
- 空欄・「要確認」が残っていないか（残っているなら、埋めるのではなく元記録を取りに行く）
- 稼働ログの合計が、利用者の体感と大きくズレていないか（ズレていれば入口の取りこぼし）

5-2. 取りこぼしの発見

- 月末に「今月やったのに記録がないこと」を利用者が5分で書き出す → 翌月の入口を改善
- 特に電話・対面・移動中の思いつきは落ちる。デイリーノート一行の習慣が守られているか

5-3. バージョン管理

- vault 全体を Git 管理（ローカルのみ、リモートに push しない）。月1回コミット
- 生成物を作り直したら、前版は Git に残る。sent/ は上書き禁止

5-4. セキュリティ

- vault はクラウド同期しない。バックアップは暗号化した外付け or ローカルNAS
- APIキーは .env、Git 管理外
- Claude API に送るのはテキストのみ（音声・原本ファイルは送らない）。学習に使われない前提（要確認・合意）
- クライアントとの契約で「業務記録の保管」が問題ないか確認する

5-5. 続けるための設計

- 利用者の作業は「録音を置く」「週1エクスポート」「月初レビュー」の3つに固定。増やさない
- 3ヶ月続いたら、tl;dv 自動連携やメール自動取得を足すか検討する
- 6ヶ月続いて資料が溜まったら、過去分の横断検索（「A社で在庫の話をしたのはいつ」）を Claude Code の質問で実現する。DBは要らない

6. 最初に作る順番

1. フォルダ構成と CLAUDE.md（生成ルール・禁止事項）
2. 02_templates/ の5テンプレート
3. inbox_sort.py（文字起こし .md と Slack エクスポートの2種だけ対応）
4. Claude Code スラッシュコマンド /monthly
5. 実際の1ヶ月分で試す → 取りこぼしと生成品質を見る
6. LINE / Gmail 取り込み、定時実行、通知は5の結果を見てから

ここまでで、Python 2本・テンプレート5枚・CLAUDE.md 1枚。それ以上を最初から作らない。